

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	M Poojitha Reddy	Reg. No.:	192472352
Section / Batch:	2024	Date:	25-08-26

Code of Conduct

/ M Poojitha Reddy - 192472352 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **M Poojitha Reddy**_____

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

Annexure A – Case Study

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- $C(\text{machine}) = 3$
- $C(\text{machine learning}) = 3$
- $C(\text{learning improves}) = 1$
- $C(\text{learning enables}) = 1$
- $C(\text{learning drives}) = 1$
- $C(\text{learning improves business}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word **"learning"** are:

- $P(\text{improves}) = 0.33$
- $P(\text{enables}) = 0.33$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **$P(\text{learning} | \text{machine})$** . Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.

3. Using Deleted Interpolation, calculate the probability of the sequence "**machine learning improves**" using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **$P(\text{improves}) = 0.33$, $P(\text{enables}) = 0.33$, $P(\text{drives}) = 0.33$** . Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.
5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. "**Book an appointment with the doctor.**"
2. "**The book contains medical information.**"

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word "**book**", the system is given:

- $P(\text{book} \mid \text{VB}) = 0.7$
- $P(\text{book} \mid \text{NN}) = 0.3$
- $P(\text{Start} \rightarrow \text{VB}) = 0.6$
- $P(\text{Start} \rightarrow \text{NN}) = 0.4$

The system must determine whether "**book**" functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to "**book**" using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of "**book**" being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.

3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	4	4	4	4	3	19	58
2	4	4	4	4	4	20	
3	4	4	4	3	4	19	

Faculty In-charge	Course Coordinator	Program Director
T Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CASE STUDY 1: N-Gram Language Modeling for an E-Commerce Search System

1. MLE of Bigram Probability $P(\text{learning} \mid \text{machine})$

The Maximum Likelihood Estimate (MLE) for a bigram is:

Therefore,

For example, if the corpus contains:

- `machine learning` = 40 times
- `machine` = 100 times

then:

So the probability is **0.4 or 40%**.

Interpretation

A high value of means that the search system has strong evidence that **“learning” is a likely word after “machine.”** Therefore, when a customer types **“machine”**, the system can rank **“learning”** highly as a next-word suggestion.

A low probability means that another word may be preferred. This is particularly useful in e-commerce search because the system can predict likely query completions such as **“machine learning,” “machine parts,” or “machine accessories.”**

2. Backoff Model for `"machine learning transforms"`

A backoff model uses a higher-order N-gram when sufficient training data exists. If the required N-gram is unseen, it backs off to a lower-order model.

For:

the trigram model would normally be used:

If this trigram is unseen, the model backs off to:

If the bigram is also unseen, it backs off further to the unigram:

Therefore:

Why is backoff useful?

Customer search queries frequently contain words or combinations that were never observed during training. A strict N-gram model would assign these unseen sequences a probability of zero.

Backoff avoids this problem by using lower-order information. Therefore, even when “**machine learning transforms**” has never appeared in the training corpus, the system can still estimate a meaningful probability using “**learning transforms**” or simply “**transforms.**”

3. Deleted Interpolation for "machine learning improves"

Deleted interpolation combines probabilities from different N-gram levels:

where:

For the sequence:

machine learning improves

the probability is:

Thus:

Example

Suppose the interpolation weights are:

and the probabilities are:

Then:

Why interpolation improves robustness

A trigram model provides detailed contextual information but can be unreliable when training data is sparse. A unigram model has less contextual information but is much less likely to be unseen.

Deleted interpolation combines all three levels, allowing the model to benefit from:

- **Trigram:** strongest contextual information

- **Bigram:** moderate contextual information
- **Unigram:** general word frequency

Therefore, the model remains robust even when some higher-order N-grams are rare or missing.

4. Entropy of the prediction distribution

Given:

Entropy is:

Therefore:

Approximately:

If the probabilities were exactly , entropy would be:

Interpretation

The entropy is close to the **maximum possible entropy for three equally likely outcomes**.

This means the search system has **high uncertainty** about which word should be predicted next. The system considers "improves", "enables", and "drives" almost equally likely.

A lower entropy would indicate that one prediction dominates and the system has greater confidence.

Case Study 1 — Complete Python Program

The following program uses **NLTK for N-gram generation**, **NumPy for probability calculations**, and **math for entropy**.

Because your question does not provide the actual corpus counts or interpolation weights, the program contains clearly marked values that you can replace with your question's values.

```
import nltk
import numpy as np
import math
from collections import Counter
from nltk.util import ngrams

# -----
# CASE STUDY 1: N-GRAM LANGUAGE MODEL
```

```

# -----

print("=" * 70)
print("CASE STUDY 1: E-COMMERCE SEARCH N-GRAM LANGUAGE MODEL")
print("=" * 70)

# -----
# 1. CORPUS
# -----

corpus = """
machine learning improves search
machine learning enables automation
machine learning drives innovation
machine learning improves recommendations
machine learning enables intelligent systems
machine learning drives modern applications
"""

tokens = corpus.lower().split()

print("\nCORPUS TOKENS:")
print(tokens)

# -----
# 2. GENERATE N-GRAMS USING NLTK
# -----

unigrams = list(ngrams(tokens, 1))
bigrams = list(ngrams(tokens, 2))
trigrams = list(ngrams(tokens, 3))

print("\nUNIGRAMS:")
print(unigrams)

print("\nBIGRAMS:")
print(bigrams)

print("\nTRIGRAMS:")
print(trigrams)

# -----
# 3. COUNT N-GRAMS
# -----

unigram_counts = Counter(tokens)
bigram_counts = Counter(bigrams)
trigram_counts = Counter(trigrams)

print("\nUNIGRAM COUNTS:")
print(unigram_counts)

print("\nBIGRAM COUNTS:")
print(bigram_counts)

print("\nTRIGRAM COUNTS:")
print(trigram_counts)

```

```

# -----
# 4. MLE BIGRAM P(learning | machine)
# -----

machine_count = unigram_counts["machine"]
machine_learning_count = bigram_counts[("machine", "learning")]

if machine_count > 0:
    mle_bigram = machine_learning_count / machine_count
else:
    mle_bigram = 0

print("\n" + "=" * 70)
print("MLE BIGRAM PROBABILITY")
print("=" * 70)

print(f"Count(machine) = {machine_count}")
print(f"Count(machine, learning) = {machine_learning_count}")

print("Formula:")
print("P(learning | machine) = Count(machine, learning) / Count(machine)")

print(f"= {machine_learning_count} / {machine_count}")
print(f"= {mle_bigram:.4f}")

# -----
# 5. BACKOFF MODEL
# -----

word = "transforms"

print("\n" + "=" * 70)
print("BACKOFF MODEL")
print("=" * 70)

# Trigram probability:
# P(transforms | machine, learning)

trigram_count = trigram_counts[("machine", "learning", word)]
machine_learning_count = bigram_counts[("machine", "learning")]

if machine_learning_count > 0:
    trigram_probability = trigram_count / machine_learning_count
else:
    trigram_probability = 0

print(f"Count(machine, learning, transforms) = {trigram_count}")
print(f"P(transforms | machine, learning) = {trigram_probability:.4f}")

if trigram_count > 0:
    backoff_probability = trigram_probability
    print("Trigram exists -> using trigram probability.")

else:
    # Back off to bigram P(transforms | learning)
    learning_count = unigram_counts["learning"]

```

```

learning_transforms_count = bigram_counts[("learning", "transforms")]

if learning_count > 0:
    bigram_probability = learning_transforms_count / learning_count
else:
    bigram_probability = 0

print(f"Count(learning) = {learning_count}")
print(f"Count(learning, transforms) = {learning_transforms_count}")
print(f"P(transforms | learning) = {bigram_probability:.4f}")

if learning_transforms_count > 0:
    backoff_probability = bigram_probability
    print("Bigram exists -> backing off to bigram probability.")

else:
    # Back off to unigram
    total_words = len(tokens)
    unigram_probability = unigram_counts[word] / total_words

    print(f"Count(transforms) = {unigram_counts[word]}")
    print(f"Total words = {total_words}")
    print(f"P(transforms) = {unigram_probability:.4f}")

    backoff_probability = unigram_probability
    print("Bigram unseen -> backing off to unigram.")

print(f"\nFinal Backoff Probability = {backoff_probability:.4f}")

# -----
# 6. DELETED INTERPOLATION
# -----

print("\n" + "=" * 70)
print("DELETED INTERPOLATION")
print("=" * 70)

# Replace these with the interpolation weights given
# in your question if different.

lambda3 = 0.50
lambda2 = 0.30
lambda1 = 0.20

print(f"lambda3 = {lambda3}")
print(f"lambda2 = {lambda2}")
print(f"lambda1 = {lambda1}")

print(f"Sum = {lambda3 + lambda2 + lambda1}")

# Probability components

# Trigram
trigram_count = trigram_counts[("machine", "learning", "improves")]

if machine_learning_count > 0:
    p_trigram = trigram_count / machine_learning_count

```

```

else:
    p_trigram = 0

# Bigram
learning_count = unigram_counts["learning"]
learning_improves_count = bigram_counts[("learning", "improves")]

if learning_count > 0:
    p_bigram = learning_improves_count / learning_count
else:
    p_bigram = 0

# Unigram
total_words = len(tokens)
p_unigram = unigram_counts["improves"] / total_words

print("\nProbability Components:")
print(f"P(improves | machine, learning) = {p_trigram:.4f}")
print(f"P(improves | learning) = {p_bigram:.4f}")
print(f"P(improves) = {p_unigram:.4f}")

# NumPy computation
interpolated_probability = (
    lambda3 * np.float64(p_trigram)
    + lambda2 * np.float64(p_bigram)
    + lambda1 * np.float64(p_unigram)
)

print("\nDeleted Interpolation:")
print(
    f"P = ({lambda3} × {p_trigram:.4f}) + "
    f"({lambda2} × {p_bigram:.4f}) + "
    f"({lambda1} × {p_unigram:.4f})"
)

print(f"Final Interpolated Probability = {interpolated_probability:.4f}")

# -----
# 7. ENTROPY
# -----

print("\n" + "=" * 70)
print("ENTROPY OF NEXT-WORD PREDICTION")
print("=" * 70)

prediction_probabilities = {
    "improves": 0.33,
    "enables": 0.33,
    "drives": 0.33
}

entropy = 0

for word, probability in prediction_probabilities.items():
    contribution = -probability * math.log2(probability)

```

```

    print(
        f"{word}: P = {probability:.2f}, "
        f"Contribution = {contribution:.4f}"
    )

    entropy += contribution

print(f"\nEntropy = {entropy:.4f} bits")

# -----
# 8. FINAL NEXT-WORD PREDICTION
# -----

best_word = max(
    prediction_probabilities,
    key=prediction_probabilities.get
)

print("\n" + "=" * 70)
print("FINAL NEXT-WORD PREDICTION")
print("=" * 70)

print(f"Candidate predictions: {prediction_probabilities}")
print(f"Predicted next word: {best_word}")

print("\nNOTE:")
print("All three words have approximately equal probabilities.")
print("Therefore, the model has high uncertainty.")

```

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

1. Penn Treebank POS tagging

Sentence 1

"Book an appointment with the doctor."

Word	POS Tag	Explanation
Book	VB	Verb because it expresses an action/request
an	DT	Determiner
appointment	NN	Noun
with	IN	Preposition
the	DT	Determiner
doctor	NN	Noun
.	.	Punctuation

Therefore:

Here, **book** is a verb because the patient is asking the chatbot to perform the action of booking an appointment.

Sentence 2

"The book contains medical information."

Word	POS Tag	Explanation
The	DT	Determiner
book	NN	Noun
contains	VBZ	Third-person singular present verb
medical	JJ	Adjective
information	NN	Noun
.	.	Punctuation

Therefore:

Here, **book** is a noun because it refers to a physical/informational object.

Key observation

The spelling of **book** does not change, but its grammatical role changes according to context.

This is exactly why POS tagging requires **contextual analysis**.

2. HMM Probability Calculation

Given:

and:

For the beginning of a sentence:

Therefore:

For NN:

Comparison:

Therefore, based **only on the supplied HMM probabilities**:

Important contextual point

The HMM favors VB because both the transition probability and emission probability are higher:

and

However, for the second sentence, the preceding word “**The**” provides strong contextual evidence that **book** is a noun. A complete HMM would use transition probabilities involving the previous tag rather than only Start.

3. Rule-Based POS Tagging vs HMM Tagging

Feature	Rule-Based Tagging	HMM/Stochastic Tagging
Method	Uses manually written linguistic rules	Uses probabilities learned from data
Context	Explicit grammatical rules	Probabilistic context
book example	Rules can identify verb after sentence start in an imperative	Uses emission and transition probabilities
Training data	Usually less dependent on corpus	Requires tagged corpus
Flexibility	Limited when unseen patterns occur	Better at handling variation
Interpretability	High	Lower
Adaptability	Requires new rules	Can retrain with new data
Limitation	Rule creation is time-consuming	Depends heavily on training data

In a healthcare chatbot

Rule-based tagging can be useful for highly predictable medical phrases and safety-critical patterns. It is transparent and easy to inspect.

HMM tagging is more flexible and can handle ambiguous words using statistical evidence. However, it can make errors when the training corpus does not adequately represent healthcare language.

A practical healthcare chatbot can combine **statistical tagging with linguistic rules and domain-specific validation**.

4. Importance of the Penn Treebank Tagset

The Penn Treebank tagset provides standardized labels such as:

- **NN** — noun
- **VB** — verb
- **VBZ** — third-person singular present verb
- **JJ** — adjective
- **DT** — determiner
- **IN** — preposition/subordinating conjunction

Standardized POS tags provide a common representation that downstream NLP components can process consistently.

Intent Detection

POS information helps distinguish action-oriented phrases such as:

"Book an appointment."

from descriptive statements such as:

"The book contains information."

Entity Identification

POS tags help identify likely entities and noun phrases, such as:

doctor, appointment, medical information.

Response Generation

The chatbot can use grammatical structure to generate more appropriate responses.

For example:

"Your appointment with the doctor has been booked."

Thus, POS tagging acts as an intermediate linguistic layer between raw user input and higher-level chatbot functions.

Case Study 2 — Complete Python Program

```

import nltk

# -----
# CASE STUDY 2: HEALTHCARE APPOINTMENT CHATBOT
# -----

print("=" * 75)
print("CASE STUDY 2: AI-POWERED HOSPITAL APPOINTMENT CHATBOT")
print("=" * 75)

# Download required NLTK resources if necessary
try:
    nltk.data.find("tokenizers/punkt")
except LookupError:
    nltk.download("punkt")

try:
    nltk.data.find("taggers/averaged_perceptron_tagger")
except LookupError:
    nltk.download("averaged_perceptron_tagger")

# -----
# 1. SENTENCES
# -----

sentence1 = "Book an appointment with the doctor."
sentence2 = "The book contains medical information."

# -----
# 2. TOKENIZATION
# -----

tokens1 = nltk.word_tokenize(sentence1)
tokens2 = nltk.word_tokenize(sentence2)

print("\nTOKENIZED SENTENCE 1:")
print(tokens1)

print("\nTOKENIZED SENTENCE 2:")
print(tokens2)

# -----
# 3. NLTK POS TAGGING
# -----

tags1 = nltk.pos_tag(tokens1)
tags2 = nltk.pos_tag(tokens2)

print("\n" + "=" * 75)
print("NLTK POS TAGGING")
print("=" * 75)

print("\nSentence 1:")
print(tags1)

print("\nSentence 2:")
print(tags2)

```

```

# -----
# 4. HMM PROBABILITIES FOR 'BOOK'
# -----

p_book_given_vb = 0.7
p_book_given_nn = 0.3

p_start_vb = 0.6
p_start_nn = 0.4

# HMM calculation

prob_vb = p_start_vb * p_book_given_vb
prob_nn = p_start_nn * p_book_given_nn

print("\n" + "=" * 75)
print("HMM PROBABILITY CALCULATION")
print("=" * 75)

print("\nFor VB:")
print(f"P(Start -> VB) = {p_start_vb}")
print(f"P(book | VB) = {p_book_given_vb}")

print(
    f"P(VB, book) = {p_start_vb} * {p_book_given_vb}"
)
print(f"P(VB, book) = {prob_vb:.4f}")

print("\nFor NN:")
print(f"P(Start -> NN) = {p_start_nn}")
print(f"P(book | NN) = {p_book_given_nn}")

print(
    f"P(NN, book) = {p_start_nn} * {p_book_given_nn}"
)
print(f"P(NN, book) = {prob_nn:.4f}")

# -----
# 5. MOST PROBABLE TAG
# -----

if prob_vb > prob_nn:
    best_tag = "VB"
else:
    best_tag = "NN"

print("\n" + "=" * 75)
print("HMM FINAL DECISION")
print("=" * 75)

print(f"Probability of VB = {prob_vb:.4f}")
print(f"Probability of NN = {prob_nn:.4f}")
print(f"Most probable tag for 'book' = {best_tag}")

# -----
# 6. CONTEXTUAL INTERPRETATION

```

```
# -----  
  
print("\n" + "=" * 75)  
print("CONTEXTUAL INTERPRETATION")  
print("=" * 75)  
  
print("\nSentence 1:")  
print("Book/VB an/DT appointment/NN with/IN the/DT doctor/NN")  
  
print("\nExplanation:")  
print("'Book' is a verb because it expresses an action.")  
  
print("\nSentence 2:")  
print("The/DT book/NN contains/VBZ medical/JJ information/NN")  
  
print("\nExplanation:")  
print("'Book' is a noun because it refers to an object.")
```

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

1. Apply the Transformation-Based Rule

Initial tags:

```
market/NN  
growth/NN  
drives/NNS  
investment/NN
```

Rule:

Change NNS to VBZ if the preceding word is tagged as NN.

Check drives:

- drives is currently NNS
- Previous word is growth
- growth is tagged NN

Therefore, the rule applies:

Corrected sequence

```
market/NN  
growth/NN  
drives/VBZ
```

investment/NN

So:

2. Why is `drives` a VBZ?

Consider:

"Market growth drives investment."

The grammatical structure is:

```
Market growth → Subject
drives        → Verb
investment     → Object
```

Here, **"market growth"** is a singular noun phrase.

Therefore, the present-tense verb must agree with the singular subject:

The correct Penn Treebank tag is:

NNS represents a plural noun, which is grammatically inappropriate in this sentence.

The preceding tag NN provides useful contextual evidence to the transformation-based tagger.

3. Relative Frequency / Probability

Given:

Word	Frequency
market	500
growth	350
drives	180
investment	420

Total frequency:

Therefore:

Market

Growth

Drives

Investment

Final distribution

Word	Frequency	Probability
market	500	0.3448
growth	350	0.2414
drives	180	0.1241
investment	420	0.2897
Total	1450	1.0000

Importance

Corpus frequency provides statistical evidence about how frequently words occur.

A POS tagger or language model can combine frequency with contextual information to determine likely interpretations. Frequent words may also have more reliable probability estimates than extremely rare words.

4. Entropy of the Word-Frequency Distribution

Entropy is:

Using the calculated probabilities:

The result is:

Before transformation

After transformation

The transformation changes:

drives/NNS

to:

drives/VBZ

It does not change the word frequencies.

Therefore:

and:

Important interpretation

This is an important distinction:

POS transformation changes linguistic interpretation, not the underlying word-frequency distribution.

Therefore, if entropy is calculated from the four word frequencies given in the question, it remains unchanged.

If entropy were instead calculated over the **POS-tag distribution**, then changing NNS to VBZ could change the entropy of that tag distribution.

Case Study 3 — Complete Python Program

```
import nltk
import math
from collections import Counter

# -----
# CASE STUDY 3
# FINANCIAL NEWS POS TAG CORRECTION
# -----

print("=" * 75)
print("CASE STUDY 3: FINANCIAL NEWS POS TAG CORRECTION")
print("=" * 75)

# -----
# 1. INPUT SENTENCE
# -----

sentence = "Market growth drives investment."

tokens = nltk.word_tokenize(sentence)

print("\nTOKENS:")
```

```

print(tokens)

# -----
# 2. INITIAL POS TAGGING
# -----

# The case study specifies these initial tags.
initial_tags = [
    ("Market", "NN"),
    ("growth", "NN"),
    ("drives", "NNS"),
    ("investment", "NN"),
    (".", ".")
]

print("\n" + "=" * 75)
print("INITIAL POS TAGS")
print("=" * 75)

for word, tag in initial_tags:
    print(f"{word}/{tag}")

# -----
# 3. TRANSFORMATION-BASED TAGGER
# -----

corrected_tags = initial_tags.copy()

for i in range(1, len(corrected_tags)):

    current_word, current_tag = corrected_tags[i]
    previous_word, previous_tag = corrected_tags[i - 1]

    # Rule:
    # Change NNS to VBZ if preceding word is NN

    if current_tag == "NNS" and previous_tag == "NN":
        corrected_tags[i] = (current_word, "VBZ")

print("\n" + "=" * 75)
print("TRANSFORMATION RULE")
print("=" * 75)

print("Rule:")
print("Change NNS to VBZ if the preceding word is tagged as NN.")

print("\n" + "=" * 75)
print("CORRECTED POS TAGS")
print("=" * 75)

for word, tag in corrected_tags:
    print(f"{word}/{tag}")

# -----
# 4. CORPUS FREQUENCIES
# -----

```

```

word_frequencies = {
    "market": 500,
    "growth": 350,
    "drives": 180,
    "investment": 420
}

total_frequency = sum(word_frequencies.values())

print("\n" + "=" * 75)
print("CORPUS FREQUENCY ANALYSIS")
print("=" * 75)

print(f"Total frequency = {total_frequency}")

probabilities = {}

for word, frequency in word_frequencies.items():

    probability = frequency / total_frequency
    probabilities[word] = probability

    print(
        f"{word:12s} "
        f"Frequency = {frequency:4d} "
        f"Probability = {probability:.4f}"
    )

# -----
# 5. ENTROPY BEFORE TRANSFORMATION
# -----

entropy_before = 0

print("\n" + "=" * 75)
print("ENTROPY BEFORE TRANSFORMATION")
print("=" * 75)

for word, probability in probabilities.items():

    contribution = -probability * math.log2(probability)

    print(
        f"{word:12s} "
        f"P = {probability:.4f} "
        f"Contribution = {contribution:.4f}"
    )

    entropy_before += contribution

print(f"\nEntropy Before = {entropy_before:.4f} bits")

# -----
# 6. ENTROPY AFTER TRANSFORMATION
# -----

# The transformation changes POS tags only.

```

```

# Word frequencies remain unchanged.

entropy_after = 0

print("\n" + "=" * 75)
print("ENTROPY AFTER TRANSFORMATION")
print("=" * 75)

for word, probability in probabilities.items():

    contribution = -probability * math.log2(probability)

    print(
        f"{word:12s} "
        f"P = {probability:.4f} "
        f"Contribution = {contribution:.4f}"
    )

    entropy_after += contribution

print(f"\nEntropy After = {entropy_after:.4f} bits")

# -----
# 7. ENTROPY DIFFERENCE
# -----

entropy_difference = entropy_after - entropy_before

print("\n" + "=" * 75)
print("ENTROPY COMPARISON")
print("=" * 75)

print(f"Entropy Before = {entropy_before:.4f} bits")
print(f"Entropy After = {entropy_after:.4f} bits")
print(f"Change = {entropy_difference:.4f} bits")

# -----
# 8. FINAL RESULT
# -----

print("\n" + "=" * 75)
print("FINAL POS TAGGING RESULT")
print("=" * 75)

for word, tag in corrected_tags:
    print(f"{word}/{tag}")

print("\nFinal sentence:")
print("Market/NN growth/NN drives/VBZ investment/NN ./.")

print("\nConclusion:")
print("The transformation rule correctly changes drives/NNS to drives/VBZ.")
print("The word-frequency entropy remains unchanged because the word")
print("frequencies themselves are not modified by the POS transformation.")

```

Final Calculation Summary

Case Study 1

Question	Final Result
MLE Bigram	
Backoff	Trigram $\rightarrow$ Bigram $\rightarrow$ Unigram
Deleted Interpolation	
Prediction Entropy	≈ 1.585 bits
Interpretation	High uncertainty because all three predictions are almost equally likely

Note: Numerical MLE, backoff, and interpolation results require the missing corpus counts/probabilities and interpolation weights from your original question.

Case Study 2

Item	Result
Book in sentence 1	VB
book in sentence 2	NN
HMM VB probability	
HMM NN probability	
HMM preferred tag	VB
Reason	Higher transition $\times$ emission probability

Case Study 3

Item	Result
Total frequency	1450
P(market)	0.3448
P(growth)	0.2414
P(drives)	0.1241
P(investment)	0.2897
Entropy before	1.9161 bits
Entropy after	1.9161 bits
Entropy change	0 bits
Corrected drives tag	VBZ

One important exam point

For Case Study 3, do not say that the entropy decreases merely because the POS tagging becomes **more accurate**. The question specifically asks for entropy of the **word-frequency distribution**. Since the transformation only changes:

and does not change $\text{market} = 500, \text{growth} = 350, \text{drives} = 180, \text{or investment} = 420$, the word-frequency entropy **remains exactly 1.9161 bits**.